SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT SIH26251

DRONA-LOG

Predictive Logistics and Terrain-Aware Resupply for High-Altitude Formations

DETAILED PROJECT REPORT

A working, end-to-end decision-support prototype for the Ladakh sector

A forecast → route → optimise → replan loop for forward supply, with an offline, hash-chained edge ledger. The prototype combines LightGBM demand models, terrain-aware A*, an OR-Tools/SCIP mixed-integer allocator, automatic pass-closure replanning and operator-visible evidence.

TEAM	Subterra Mesh
TEAM ID	185972
PROBLEM STATEMENT	SIH26251 — Predictive Logistics and Forward Supply Chain
TECHNOLOGY BUCKET	Software
PROTOTYPE STATUS	Working end-to-end software; Docker Compose deployment
REPOSITORY	github.com/athul-dev-sys/DRONA-LOG
DOCUMENT	Version 1.0 · 20 October 2026 · Final for submission
DATA	Synthetic; every operational figure is labelled SIMULATED

Prepared for Smart India Hackathon 2026. This is a prototype on synthetic data and is not an operational deployment.

SUBMISSION RECORD

Document control

This report documents the design, implementation, evidence and limitations of the DRONA-LOG prototype submitted against problem statement SIH26251. It is written to be read alongside the running repository, the six-page idea template and the recorded demonstration.

Field	Value
Document title	DRONA-LOG — Detailed Project Report
Version / status	1.0 · Final for submission
Date	20 October 2026
Prepared by	Team Subterra Mesh, Team ID 185972
Problem statement	SIH26251 — Predictive Logistics and Forward Supply Chain for the Indian Army in the Ladakh Sector
Technology bucket	Software
Primary artefact	Working backend, frontend, optimisation and edge-ledger prototype
Companion artefacts	6-page idea template (PDF), recorded demo (MP4), submission answers (MD)
Repository	github.com/athul-dev-sys/DRONA-LOG
Data basis	Synthetic seed 26251; no live, personal or classified data

Revision history

Version	Date	Author	Change
1.0	20 Oct 2026	Team Subterra Mesh	Initial detailed report for SIH submission

Evidence basis

The report is based on the repository at the submitted revision, a seeded SQLite database, the completed 30-day evaluation run, the recorded closure events, and screenshots captured from the running prototype. The backend and frontend test suites were executed on 20 October 2026. The key measured artefacts are:

- 24 backend tests passed with `python -m pytest tests/ -q` (6.84 s in the recorded run).
- 6 frontend ledger tests passed with `npx vitest run` (528 ms in the recorded run).
- Production frontend bundle built successfully with `npm run build`.
- Closure events recorded on 20 October 2026: 28.5 ms reroute and 688.5 ms re-solve on closure; 20.7 ms and 517.0 ms on reopen.
- 30-day simulation counters: 755 units of demand, Zoji La closed on days 10–16, naive and DRONA policies evaluated on the same seed.

A note on numbers

SIMULATED — recomputed, not baked in

Every operational number in this report is produced by the seeded simulation or by a live solve at runtime. Solve times are wall-clock measurements from the development machine and will differ across hardware. The 30-day counters are deterministic for the fixed seed and configuration, but a time-limited SCIP solve can change the last decimal of a plan objective on another machine. Where a value is a trade-off rather than a benefit, it is labelled as such.

NAVIGATION

Contents

Executive Summary 6
What the prototype demonstrates 6
Submission position 6
1. Context and Problem Definition 7
1.1 Operational setting 7
1.2 Why the conventional process fails 7
1.3 Problem statement 7
1.4 Objectives and success criteria 7
1.5 Scope, assumptions and non-goals 8
2. Solution Overview 9
2.1 Concept of operations 9
2.2 Capability map 9
2.3 Design principles 9
2.4 Users and workflows 10
3. System Architecture 11
3.1 Component view 11
3.2 Backend services 11
3.3 Operator frontend 11
3.4 Data layer 12
3.5 Deployment topology 12
3.6 Request lifecycles 12
4. Synthetic Theatre and Data 13
4.1 Why synthetic data 13
4.2 Network and formations 13
4.3 Demand, weather and history 13
4.4 Data dictionary 13
4.5 Determinism and reproducibility 14
5. Demand Forecasting 15
5.1 Formulation 15
5.2 Features 15
5.3 Model configuration 15
5.4 Training and inference pipeline 15
5.5 Output contract 16
5.6 Limitations 16
6. Terrain-Aware Routing 17
6.1 Graph model 17
6.2 Elevation-penalised cost 17

6.3 A* and the route matrix 17
6.4 Closure handling 17
6.5 Worked corridor example 17
6.6 Limitations 17
7. Allocation Optimisation 19
7.1 Decision variables 19
7.2 Objective function 19
7.3 Constraints 19
7.4 Solver configuration 20
7.5 Plan outputs 20
7.6 Baseline plan behaviour 20
7.7 Limitations 20
8. Closure Response 21
8.1 Event pipeline 21
8.2 Recorded closure run 21
8.3 Plan differences 22
8.4 Why the objective rises 22
8.5 Operator experience 22
8.6 Reopening and multiple closures 22
9. Edge Operations and Ledger Integrity 23
9.1 Offline-first design 23
9.2 Hash-chained ledger 23
9.3 Sync verification 23
9.4 Rejection taxonomy 23
9.5 Demonstrated tamper rejection 24
9.6 Stock semantics and limitations 24
10. User Experience and Interface 25
10.1 Command view 25
10.2 Visual system 25
10.3 States and feedback 26
10.4 Accessibility and usability notes 26
11. Evaluation 27
11.1 Methodology and fairness controls 27
11.2 Results 27
11.3 Interpretation and trade-offs 28
11.4 Variability 29
11.5 Threats to validity 29
12. Acceptance Criteria and Evidence 30
12.1 Criterion-by-criterion evidence 30
12.2 Test and demonstration inventory 30

13. Testing and Quality Assurance 31
13.1 Strategy 31
13.2 Backend suite 31
13.3 Frontend suite 31
13.4 Determinism and reproducibility 31
13.5 Coverage gaps 31
14. Deployment and Operations 32
14.1 Container topology 32
14.2 First boot 32
14.3 Local development 32
14.4 Reset and troubleshooting 32
14.5 Runtime dependencies 33
15. Security, Ethics and Data Handling 34
15.1 Current security posture 34
15.2 Gaps before production 34
15.3 Data ethics and responsible claims 34
16. Limitations and Roadmap 35
16.1 Current limitations 35
16.2 Production roadmap 35
16.3 What would make the evaluation stronger 35
17. Conclusion 36
References 37
Appendix A — API Reference 38
Appendix B — Configuration Parameters 39
Appendix C — Test Inventory 40
C.1 Backend (24 tests) 40
C.2 Frontend (6 tests) 40
Appendix D — Reproducibility Checklist 41
Appendix E — Submission Artefacts 42

Page numbers and the table of contents are updated in Microsoft Word and are regenerated in the exported PDF. Chapter numbering is manual so that the document remains readable even if fields are not refreshed.

Executive Summary

Forward supply in high-altitude sectors is a constrained decision problem disguised as a transport problem. A small number of passes carry most of the tonnage; weather and road conditions can remove one for days; demand moves with troop strength, season and exercises; and the fleet at each depot is finite. A plan that ignores terrain, corridor capacity or fleet limits is not a plan — it is a wish list. DRONA-LOG is a working prototype that makes the decision loop explicit and fast: forecast demand, route over the terrain, allocate under capacity, and replan when a pass closes. It also keeps requisitions working at the edge when the network does not.

The prototype is deliberately end-to-end rather than a collection of notebooks. A FastAPI backend exposes a seeded synthetic theatre, a LightGBM forecast per supply class, an elevation-penalised A* router, an OR-Tools/SCIP mixed-integer allocator, a closure-event pipeline and a server-verified edge ledger. A React command view presents the map, the 7-day plan, stock versus buffer and the 30-day impact comparison; a separate edge client records offline requisitions in IndexedDB. Docker Compose brings the whole stack up with one command, and the first boot trains the three models and seeds the database.

What the prototype demonstrates

Capability	Observed result in the seeded run
7-day baseline plan	OPTIMAL; 15 truck dispatches; 0 stockout units; 0 buffer-deficit units; objective ₹23.14 lakh
Pass closure	Zoji La removed; west gateway route unreachable; south route via Manali active; 28.5 ms reroute and 688.5 ms re-solve in the recorded event
Plan change	6 deliveries cancelled, 5 added, 4 modified, 5 unchanged; closure objective ₹27.94 lakh
Offline edge ledger	Entries recorded with the backend stopped, accepted after restart; hand-edited entry rejected as HASH MISMATCH
30-day comparison	Stockouts 4.15 → 0.25 units; below-buffer unit-days 983.9 → 493.3; cost per unit ₹30,242 → ₹25,619
Test suites	24 backend tests and 6 frontend ledger tests passing; production frontend build succeeds

The 30-day comparison is presented with its trade-offs. DRONA-LOG uses the longer southern corridor during the closure, so truck-kilometres per unit of demand rise by 22.1%. That is an operating cost, not a saving, and the interface presents it as a warning. The held-requisition counter of 258 for the naive policy and 0 for DRONA is also annotated: the DRONA simulation has no hold path, so the zero is structural and is not claimed as a measured queue reduction. This discipline matters more than a flattering slide: a planner needs to know what a plan costs, not only what it saves.

Submission position

The prototype meets every acceptance criterion set for the build: a feasible baseline with zero modelled stockouts, a valid alternate route and new plan after closure, displayed solve times, offline sync after restart, rejection of a hand-edited ledger entry, one-command bring-up, computed rather than hardcoded metrics, and complete run/demo documentation. It is not a production system: the data is synthetic, the network is schematic, authentication and device signing are absent, and the evaluation is a single 30-day scenario. Section 16 states those limits directly and sets out what a pilot would require.

1. Context and Problem Definition

1.1 Operational setting

The prototype models a high-altitude theatre inspired by Ladakh. Supply enters through two gateways, west and south, and reaches a base depot before moving to staging depots and forward operating bases (FOBs). The synthetic network contains 49 nodes and 53 road edges: 2 gateways, 22 transit nodes, 12 named passes, 2 staging depots, 1 base depot and 10 FOBs. The road topology is deliberately sparse, because that sparsity is the operational problem. When a pass is removed, an entire corridor can disappear and the remaining route can be hundreds of kilometres longer.

Demand is not uniform. Supply class I covers rations and fresh items, class III covers petroleum, oils and lubricants, and class V covers ammunition. Consumption responds to troop strength, season, temperature and snowfall, and can spike during exercises. Stocks at FOBs are bounded by storage and by the need to keep a safety buffer; depots also hold a war reserve. The fleet is finite and shared across destinations, so a plan that sends too many trucks to one FOB can starve another. These are the ingredients of a capacitated, time-phased, terrain-dependent allocation problem.

1.2 Why the conventional process fails

A spreadsheet-and-experience process is not irrational; it is a reasonable response to missing tools. Its failure modes are predictable:

- **Trailing demand.** Reordering from a trailing mean reacts to the last crisis rather than the next one. A forecast that uses weather and troop strength can move supply before consumption rises.
- **Nearest-depot rules.** The nearest depot is not always the cheapest or the most feasible source once gradient, corridor capacity and stock are considered. A slightly longer route can be cheaper in cost-km and less exposed to a single pass.
- **Fixed lead times.** A three-day lead with no lead-time demand compensation systematically under-orders when consumption is rising and over-orders when it falls.
- **Corridor blindness.** When a pass closes, a static plan becomes invalid instantly. Holding convoys at the closure is safe in the short term but converts a transport problem into a stockout problem.
- **Depot hoarding.** Without visibility of FOB buffers and depot reserves in one model, each echelon protects itself. Total inventory rises while availability at the edge does not.
- **Unverifiable edge data.** Paper or local records captured during an outage are hard to reconcile later. Without a tamper-evident chain, the server cannot distinguish a legitimate late entry from an altered one.

1.3 Problem statement

> **Decision problem addressed by DRONA-LOG**
>
> Given a road network with pass closures, a multi-day demand forecast, depot and FOB stocks, safety buffers, a finite fleet at each depot, corridor inflow caps and a cost model, produce a feasible dispatch plan that meets demand, maintains buffers where possible and minimises total cost — and recompute that plan within seconds when a pass closes. At the edge, capture requisitions while disconnected and later prove to the server that each entry is intact and belongs to a valid chain.

The problem has two outputs that must agree: a plan a human can act on, and evidence that the plan and its exceptions are internally consistent. A map that disagrees with the plan table, or a sync that accepts an altered entry, fails the operational test even if the optimiser is sophisticated.

1.4 Objectives and success criteria

ID	Objective	Success criterion	Where verified
O1	Produce a feasible 7-day plan	OPTIMAL or FEASIBLE; zero modelled stockouts and zero buffer deficit on the seeded baseline	Planner tests; baseline plan payload
O2	Respond to a pass closure	Severed corridor returns no route; alternate route is valid; new plan solves and is diffed	Zoji La and Khardung La tests; closure event

ID	Objective	Success criterion	Where verified
O3	Expose solve performance	Router and solve times returned and rendered; no hardcoded timing	Closure payload; UI panel
O4	Operate offline	Entries recorded with backend stopped; accepted after restart; stock applied for receipts	Sync-after-restart test; live edge demo
O5	Detect tampering	Hand-edited, replayed, gapped or broken-chain entries rejected with a reason	Eight sync tests; UI tamper demo
O6	Measure honestly	Naive and DRONA policies simulated on the same seed; all cards labelled SIMULATED	Metrics tests; 30-day report
O7	Make it reproducible	One command brings up the stack; tests and seed are deterministic	Docker Compose; test suites

1.5 Scope, assumptions and non-goals

In scope	Out of scope (prototype)
Synthetic Ladakh-inspired theatre with 49 nodes, 53 edges, 10 FOBs and 12 passes	Live weather, road reports, service records or classified feeds
Three supply classes (I, III, V), three depots, two gateways	Unit-level inventory, lot tracking, shelf-life or hazardous-materials rules
7-day operational MILP and 30-day policy comparison	Multi-theatre campaign planning or strategic force flow
Single-pass closure and reopen workflow, with a persisted set of closed passes	Simultaneous multi-pass stress testing and recovery playbooks
Offline requisitions with a per-device hash chain	Device registration, signing, PKI, conflict resolution between devices
Two operator views and OpenAPI documentation	Authentication, roles, audit administration, mobile field app

The prototype is a decision-support system. It does not fire weapons, task real vehicles or transmit classified information. All operational figures are synthetic and labelled SIMULATED; all recommendations are subject to human review.

2. Solution Overview

2.1 Concept of operations

DRONA-LOG is organised around a daily planning cycle and an exception path. The cycle is intentionally short enough to run before a morning briefing; the exception path is short enough to run while a closure is being confirmed.

Step	Operator / system action	Output
1. Sense	Load the seeded theatre, stock, weather forecast and troop strength	Current state snapshot
2. Forecast	Run one LightGBM model per class over the 7-day horizon	Daily demand by FOB and class
3. Route	Build the closure-aware graph and compute depot→FOB cost-km	Route matrix with gradient penalties
4. Allocate	Solve the capacitated MILP for trucks, loads and gateway inflow	7-day dispatch plan and stock projection
5. Publish	Return plan, routes, KPIs, timings and SIMULATED comparison in one payload	Command view
6. Except	On a pass closure, remove the pass, reroute, re-solve, diff and record	New plan, timings, diff, event
7. Reconcile	Sync edge entries; verify hash, sequence and previous link; apply receipts	Accepted/rejected ledger response

2.2 Capability map

Capability	Mechanism	Operator-visible output
Anticipate demand	LightGBM per supply class; temperature, snowfall, troops, day-of-week	7-day forecast per FOB and class
Choose realistic routes	A* on distance + 6 × gradient%; pass nodes removed on closure	Route, distance, cost-km, gradient penalty, max elevation
Allocate scarce capacity	OR-Tools/SCIP MILP with trucks, fleet caps, stock, buffers and inflow caps	Dispatch table, stock projection, objective, solve time
React to a closure	Persist closure, rebuild graph, remap routes, re-solve, diff plans	Red corridor, alternate route, timings, cancelled/added/changed counts
Operate at the edge	IndexedDB append-only ledger with SHA-256 chain	Offline status, local entries, sync result
Reconcile safely	Server recomputes hashes and checks chain continuity	Accepted/rejected entries with explicit reasons
Measure honestly	Naive vs DRONA on the same seeded 30-day scenario	SIMULATED impact cards and trade-off warnings

2.3 Design principles

Compute, do not hardcode. Every operational number is derived at runtime: plans from the MILP, routes from the graph, metrics from the simulation, timings from the solver. The UI labels data SIMULATED rather than presenting decorative numbers as facts.

One payload, one truth. The closure response contains the new routes, the new plan, the diff and the timings. The map, plan table, inventory chart and event log all read from that response or from the persisted plan, so they cannot tell different stories.

Degrade, do not stop. The edge client keeps recording when the backend is unreachable. On reconnect it syncs rather than discarding local work, and it never silently accepts an entry the server rejects.

Expose the trade-off. A longer corridor can preserve availability while increasing truck-kilometres. The interface reports the increase instead of hiding it. A zero hold counter that is structural is annotated as structural.

Be reproducible. Seed 26251, deterministic model parameters, one-thread SCIP and a fixed scenario make the demonstration repeatable. The stack needs no external API key, tile server or GPU.

2.4 Users and workflows

User	Primary questions	View and workflow
Commander / staff officer	Is the theatre covered? What does a closure do to cost and availability?	Command view: map, 7-day plan, impact panel, closure controls
Logistics planner	Which trucks, from where, with what load, on which day? What changed?	Command view: dispatch table, stock vs buffer, closure diff and timings
Edge quartermaster	Can I record a requisition without connectivity? Did it sync? Was anything rejected?	Edge client: requisition form, local chain, sync result, tamper simulation

There are two operator views, not three: the command view at `/` combines the commander and planner workflows because they read the same plan; the edge client at `/edge` serves the disconnected quartermaster. FastAPI's interactive OpenAPI documentation at `/docs` is a developer surface, not a third operational screen.

3. System Architecture

3.1 Component view

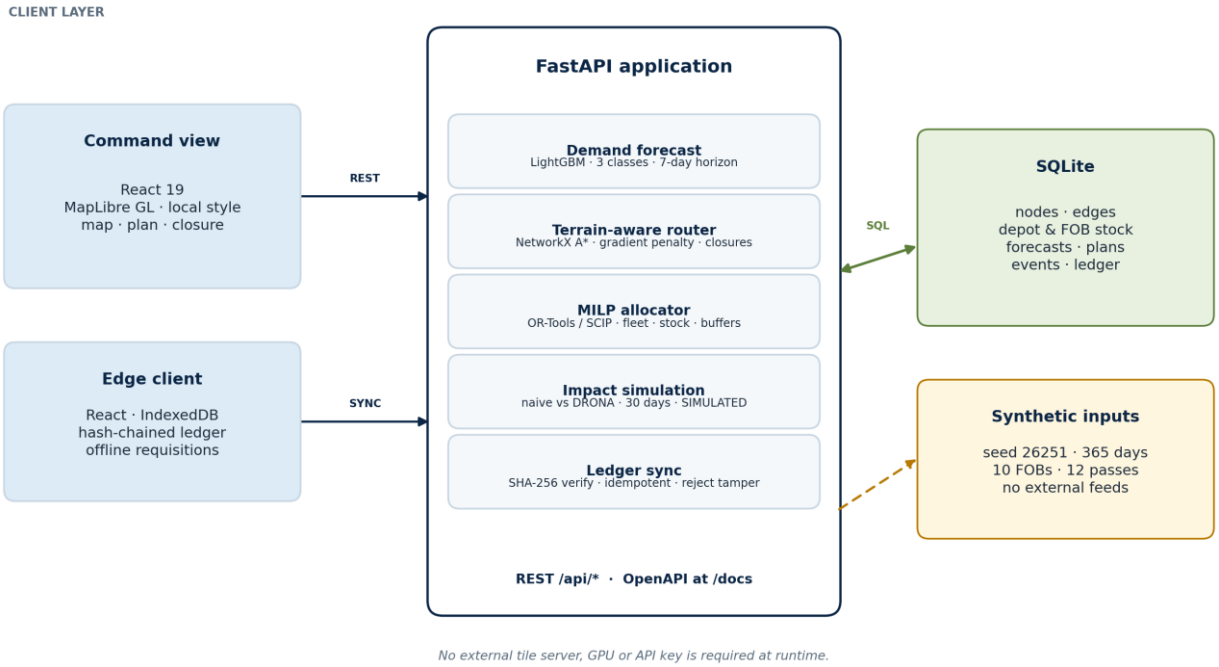


Figure 3.1 — Component architecture. Two React clients call one FastAPI application; SQLite is the single persistent store; synthetic inputs are generated locally.

The architecture separates decision engines from transport concerns. The React clients never compute a route or a plan; they render server responses. The FastAPI application owns the forecast, graph, MILP, simulation and ledger verification. SQLite stores the seeded theatre, models' inputs, plans, events and ledger records. No runtime component calls an external service.

3.2 Backend services

Module	Responsibility	Key output
`seed.py`	Builds the synthetic theatre, weather, history, stock and actual-future demand	Deterministic SQLite database
`graph.py`	Builds the NetworkX graph; removes closed pass nodes and incident edges	Closure-aware graph
`router.py`	A* over cost-km; route matrix; gateway routes; per-leg gradient detail	Routes with distance, cost-km and penalty
`forecast.py`	Trains one LightGBM model per class; predicts from weather forecast	Per-FOB, per-class daily demand
`allocator.py`	Builds and solves the MILP with OR-Tools/SCIP; returns plan and KPIs	Trucks, loads, inflow, stock projection
`scenario.py`	Assembles demand, stock, buffers, route costs and corridor caps for a closure state	Solver-ready inputs
`simulate.py`	Runs naive and DRONA policies over the same 30-day seeded scenario	SIMULATED comparison counters
`main.py`	FastAPI endpoints, closure flow, metrics cache and ledger sync verification	REST API

3.3 Operator frontend

The frontend is a React 19 single-page application built with Vite 8 and styled with Tailwind CSS 4. It has two routes: the command view and the edge client. MapLibre GL 6 renders a local, schematic theatre style; there is no external tile request, so the demonstration works without internet access. The production container

uses nginx to serve the built assets, provide SPA fallback and proxy `/api` to the backend. The development server proxies `/api` to port 8000 in the same way.

The command view is desktop-first and intentionally dense. The map occupies the flexible left pane; a 420 px right pane holds the closure impact panel, the 30-day comparison, the 7-day allocation table and FOB stock bars. The edge client uses a form-and-table layout with a 340 px sync panel. The visual language is a dark tactical surface with green for healthy/open, cyan for the southern corridor, amber for warnings and planned convoys, and red for closures or rejection.

3.4 Data layer

SQLite is sufficient for the prototype because the workload is read-mostly and single-node. The schema separates reference data, time series, plans and audit records:

Table group	Examples	Purpose
Theatre	`nodes`, `edges`, `stock`	Topology, elevations, corridors, FOB/depot stock and buffers
Time series	`history`, `weather_forecast`, `actual_future`	365-day training history, 7-day weather inputs, 30-day evaluation demand
Decisions	`plans`, `events`	Persisted plan payloads and closure/reopen events with timings and diffs
Edge trust	`ledger`, `chain_heads`, `sync_log`	Accepted entries, per-device chain head, sync audit
Metadata	`kv`	Seed flags and the persisted set of closed passes

The database file is mounted on the `drona-data` Docker volume, so plans and ledger entries survive container restarts. Rebuilding the volume returns the demonstration to a clean seeded state.

3.5 Deployment topology

Service	Image / build	Port	Role and health
backend	`backend/Dockerfile`	8000	FastAPI + seed + models; healthcheck calls `/api/health`; 12 retries over a 10 s interval
frontend	`frontend/Dockerfile`	8080 → 80	nginx serving the SPA and proxying `/api`; starts only after backend is healthy
volume	`drona-data`	—	Persistent SQLite database and trained model artefacts

The stack has no external runtime dependency. The base images, Python packages and npm packages are the only downloads, and they are fetched at build time. Runtime API keys, tile servers, CDNs and GPUs are not required.

3.6 Request lifecycles

Plan request. `GET /api/plan` checks that the database is seeded and the models exist; on first use it seeds and trains. It assembles the current scenario, computes the route matrix, solves the MILP, persists the plan and returns the plan payload with routes, gateway routes, stock projection and KPIs. The command view calls it on load and exposes a refresh action.

Closure request. `POST /api/events/pass-closure` accepts a pass id, an action and a duration. It validates the pass, updates the persisted closed-pass set, rebuilds the graph, remaps the route matrix, re-solves the plan, diffs the old and new deliveries, stores the event with both plans and timings, and returns the closure payload used by the UI.

Sync request. `POST /api/sync` accepts a device id and a batch of entries. It reads the per-device chain head, sorts by sequence, recomputes each hash, validates continuity and idempotency, inserts accepted entries, updates the chain head, applies accepted receipt quantities to stock and records a sync audit row.

4. Synthetic Theatre and Data

4.1 Why synthetic data

A hackathon prototype cannot use classified movement records, live road reports or personal data. Synthetic data is therefore a design choice, not a shortcut: it lets the team publish the scenario, reproduce every number and avoid implying access to operational information. The trade-off is external validity. The relationships in the data are plausible but constructed, so the measured improvements demonstrate that the decision loop works on a controlled problem; they do not predict performance on a real road network.

> **Data honesty rule**
> Every operational surface is labelled SIMULATED. No figure in the UI is a claimed real-world measurement. The README, this report and the demo state that the theatre, weather, demand and costs are synthetic.

4.2 Network and formations

Entity	Count	Detail
Nodes	49	2 gateways, 22 transit, 12 passes, 2 staging depots, 1 base depot, 10 FOBs
Road edges	53	Undirected; distance, simulated gradient, elevation gain, corridor
Gateways	2	Srinagar (west) and Manali (south)
Depots	3	Leh (base), Karu and Upshi (staging)
FOBs	10	DBO, Turtuk, Sasoma, Diskit, Durbuk, Spangmik, Chushul, Nyoma, Hanle, Demchok
Passes	12	Zoji La, Namika La, Fotu La, Rohtang, Baralacha, Tanglang La, Khardung La, Wari La, Saser La, Chang La, Kaksang La, Umling La

FOB names and pass names are drawn from Ladakh for recognisability. Coordinates and elevations are a simplified synthetic model, not an authoritative operational map. The network is sparse by design: the western gateway passes through Zoji La, Namika La and Fotu La; the southern gateway passes through Rohtang, Baralacha and Tanglang La. Removing Zoji La therefore severs the western corridor rather than merely lengthening it.

4.3 Demand, weather and history

The seed generates 365 days of daily history for each of 10 FOBs and 3 supply classes: 10,950 rows. Daily temperature and snowfall follow a seeded seasonal pattern; troop strength includes periodic exercise surges and small monthly drift. Consumption is driven by troop strength and weather with day-of-week variation and noise. A separate 30-day `actual_future` series (900 rows) is used only for evaluation, so the forecast and the evaluation target are not the same series. A 7-day weather forecast table supplies inference features.

Data set	Rows	Use
`history`	10,950	Train one LightGBM model per class (365 days × 10 FOBs × 3 classes)
`actual_future`	900	30-day evaluation demand (30 × 10 × 3); never used for training
`weather_forecast`	370	Inference features for the forecast horizon
`stock`	39	Initial on-hand and buffer for 10 FOBs + 3 depots × 3 classes

The evaluation closure is fixed: Zoji La is closed from day 10 to day 16 inclusive, seven days. Both policies see the same closure and the same actual demand. This is what makes the naive-versus-DRONA comparison meaningful even though the underlying data is synthetic.

4.4 Data dictionary

Field	Type / unit	Meaning
`nodes.kind`	enum	gateway, transit, pass, base depot, staging, FOB

Field	Type / unit	Meaning
`nodes.elev_m`	metres	Synthetic node elevation used by the gradient penalty
`edges.distance_km`	km	Road distance (1.25 × haversine, minimum 3 km)
`edges.gradient_pct`	percent	|Δelevation| / (distance_km × 10)
`history.qty`	units/day	Synthetic consumption for a FOB, class and date
`stock.on_hand` / `buffer`	units	Current stock and safety buffer / war reserve
`plans.payload_json`	JSON	Persisted plan, routes, projection and KPIs
`ledger.payload_json`	JSON	Canonical edge payload: FOB, class, qty, note, timestamp

4.5 Determinism and reproducibility

The seed is `26251`. Per-FOB random streams are derived from a CRC32 of the FOB id, so adding a FOB does not reshuffle the whole theatre. LightGBM is configured with `random_state=26251`, `deterministic=True`, `force_col_wise=True` and one thread. SCIP runs with one thread. These choices make a rerun on the same machine reproduce the same data and the same model. They do not make solve time a constant: SCIP is time-limited, and wall-clock time depends on the host. The report therefore quotes timings as recorded samples and the UI displays live values.

5. Demand Forecasting

5.1 Formulation

For each FOB f, supply class c and day t in the horizon, the system estimates daily consumption ŷ(f,c,t). One model is trained per class on the pooled daily history across FOBs. The forecast horizon in the command view is seven days; the allocator consumes the same daily vector. The model is a point forecast: it produces a single expected quantity, not a distribution.

```
ŷ(f, c, t) = f_c( temp_c(t), snowfall_cm(t), troops(t), dow(t) )

c ∈ {I, III, V}    t ∈ {0, …, 6}    ŷ ≥ 0
```

5.2 Features

Feature	Unit	Operational rationale
`temp_c`	°C	Cold-weather demand and storage conditions
`snowfall_cm`	cm	Access difficulty, heating and clothing consumption
`troops`	persons	Scale of demand at the FOB
`dow`	0–6	Day-of-week pattern in issuing and consumption

The model is deliberately compact. A larger feature set would require data the prototype does not have, and would make the forecast harder to explain to a planner. The features are all present in both the historical table and the weather forecast table, which keeps training and inference consistent.

5.3 Model configuration

Parameter	Value	Reason
Algorithm	LightGBM `LGBMRegressor`	Fast, strong on tabular data, no GPU required
Objective	`regression` (L2)	Daily quantities are continuous and roughly symmetric
Trees	300	Enough capacity for 365 days × 10 FOBs per class
Learning rate	0.05	Stable fit with 300 trees
Leaves	31	Moderate non-linearity without overfitting
Min child samples	20	Regularisation on small daily counts
Random state	26251	Reproducible
Threads	1	Determinism over speed for a small data set

5.4 Training and inference pipeline

At first boot the backend checks for three persisted model files. If any is missing it trains all three from `history`, writes them to the model directory and continues. The health endpoint reports readiness. Inference reads the `weather_forecast` rows for the requested dates, builds the same four features, predicts per class and clamps negative outputs to zero. The result is shaped into a per-FOB, per-class payload with daily values and a total.

Stage	Input	Output
Train	`history` rows for one class	Fitted LightGBM model
Persist	Fitted model	`data/models/model_<class>.pkl`
Infer	`weather_forecast` for horizon	Daily demand per FOB and class
Clamp / round	Raw prediction	Non-negative value, 4 decimal places
Shape	Predictions	`{fob: {class: {daily, total}}}` payload

5.5 Output contract

The forecast endpoint returns dates, FOB metadata and per-class daily values. The allocator consumes the same daily vectors. A shortened example:

```
{
  "start_date": "2026-10-01",
  "dates": ["2026-10-01", "..."],
  "fobs": {
    "DBO": {
      "name": "Daulat Beg Oldi",
      "troops": 420,
      "forecast": {
        "I":   {"daily": [2.31, 2.28, "..."], "total": 16.1},
        "III": {"daily": [5.02, 5.11, "..."], "total": 35.4},
        "V":   {"daily": [0.74, 0.70, "..."], "total": 5.1}
      }
    }
  }
}
```

The numbers above are illustrative of the shape, not a claimed sample from a particular run. The live payload is the source of truth.

5.6 Limitations

- **Point forecast.** There is no prediction interval, so the MILP cannot optimise against a demand distribution or a service level. Buffers and penalties are the current risk mechanism.
- **Pooled model.** One model per class is trained across all FOBs; FOB identity is not a feature. Distinct local patterns must be explained by troops and weather, which is a simplification.
- **Synthetic relationships.** The data generator encodes plausible effects, not observed ones. Forecast accuracy on this data is not evidence of accuracy on service records.
- **No retraining schedule.** Models train at first boot or when files are missing. A production system would need drift monitoring and scheduled retraining.
- **Uncertainty in weather.** The weather forecast is treated as exact. A real system would carry forecast error into the demand model or the optimisation.

6. Terrain-Aware Routing

6.1 Graph model

The road network is an undirected NetworkX graph. Each node carries a kind, coordinates and elevation. Each edge carries distance, elevation gain, gradient percentage and corridor. The graph is rebuilt for every scenario so that closures cannot leak between requests. A closed pass is removed as a node, which removes every incident edge at once; this is more robust than marking edges closed, because it prevents a route from transiting the pass under any combination of incoming and outgoing edges.

6.2 Elevation-penalised cost

Edge cost is expressed in cost-kilometres, not minutes or rupees. The conversion factor is deliberately simple and explainable:

```
gradient_pct = |Δelev_m| / (distance_km × 10)
cost_km      = distance_km + 6.0 × gradient_pct
```

A 10 km edge with a 5% gradient costs 40 cost-km: 10 km of distance plus 30 km of climb penalty. A flat 12 km edge costs 12 cost-km and wins. The multiplier `6.0` is a policy parameter, not a measured fuel model; it makes the router prefer lower-gradient routes without pretending to be a physics simulation. The route response reports both `distance_km` and `gradient_penalty_km`, so a planner can see why a longer route was chosen.

6.3 A* and the route matrix

A* uses a great-circle heuristic to the destination. The heuristic is admissible because the true cost of a path is at least its great-circle distance; the gradient penalty only adds cost. The router computes a matrix from each of the three depots to every FOB and the two staging depots. Unreachable pairs are returned as `None` and excluded from the MILP, so the optimiser cannot dispatch to a severed destination. The API also exposes the two gateway routes — Srinagar to Leh and Manali to Leh — for the map and the closure panel.

6.4 Closure handling

`build_graph(conn, closed_passes)` skips closed pass nodes and any edge whose endpoint is missing. The route matrix is recomputed from scratch. For a Zoji La closure, the western gateway route becomes `None`; the southern route remains and avoids Zoji La. For a Khardung La closure, Nubra traffic is routed via Wari La, which is verified by a dedicated test. The allocator receives only the reachable pairs and the corridor caps for the current state.

6.5 Worked corridor example

Gateway route	Distance	Cost-km	Gradient penalty	Max elevation	Status
Srinagar → Leh (west)	377.06 km	517.51	140.45 km	4,108 m	Open baseline; None when Zoji La is closed
Manali → Leh (south)	331.36 km	519.59	—	—	Open baseline and closure alternate

The southern route is 45.7 km shorter but slightly more expensive in cost-km because its climbs attract a larger gradient penalty. It also has lower daily inflow caps and a 33% higher procurement cost. Those differences are why a closure changes the plan rather than merely replacing one road with another.

6.6 Limitations

- **Schematic geometry.** Path geometry is a sequence of node coordinates, not a surveyed road alignment. Distance is 1.25 × haversine with a 3 km minimum, not a routing-engine result.
- **Static gradients.** Each edge has one gradient value. Seasonal road quality, surface, switchbacks and direction-dependent ascent are not modelled.
- **No traffic or weather delays.** Travel time is not modelled; the lead time is a fixed one day for every route.

- **One penalty weight.** The gradient multiplier is a single policy parameter. A production model would calibrate it against fuel, time and maintenance data.

7. Allocation Optimisation

7.1 Decision variables

Variable	Domain	Meaning
`x[d,j,c,t]`	continuous ≥ 0	Units of class c dispatched from depot d to destination j on day t; arrives t+1
`n[d,j,t]`	integer ≥ 0	Trucks dispatched from d to j on day t
`s[f,c,t]`	continuous ≥ 0	FOB stock at start of day t
`short[f,c,t]`	continuous ≥ 0	Unmet demand (stockout) at FOB f, class c, day t
`bdef[f,c,t]`	continuous ≥ 0	Units below the FOB safety buffer on day t
`sd[d,c,t]`	continuous ≥ 0	Depot stock at start of day t
`in_w[c,t]`, `in_s[c,t]`	continuous, capped	Gateway inflow from west and south on day t
`bdef_d[d,c]`	continuous ≥ 0	End-of-horizon war-reserve shortfall at depot d

7.2 Objective function

The objective is a sum of transport, procurement and soft-penalty terms. It is denominated in rupees, so a stockout penalty of ₹500,000 per unit dominates ordinary transport cost and makes stockouts a last resort rather than an equal trade.

Term	Coefficient / formula	Purpose
Transport variable	₹42 × cost-km × units	Distance and gradient cost of moving each class
Transport fixed	₹4,000 × trucks	Dispatch overhead per truck
Procurement west	₹18,000 × west inflow	Gateway supply cost via Zoji La corridor
Procurement south	₹24,000 × south inflow	33% premium for the longer Manali corridor
Stockout	₹500,000 × unmet units	Strongly discourage unmet demand
FOB buffer deficit	₹60,000 × unit-days below buffer	Protect safety stock
Depot reserve deficit	₹60,000 × end-horizon shortfall	Discourage consuming the war reserve
Day tie-break	₹40 × day × inflow	Break procurement-timing symmetry

7.3 Constraints

Constraint	Formulation / effect
Truck capacity	Load on a route ≤ 10 × trucks, with classes sharing the truck
Fleet per depot per day	Sum of trucks from Leh/Karu/Upshi ≤ 24 / 8 / 8
Delivery lead	Dispatch on day t arrives at the destination on day t+1
FOB stock balance	Start stock + arrivals − demand + stockout = next start stock
FOB buffer	Start stock + buffer deficit ≥ 7-day buffer (soft)
Depot stock balance	Start stock + inflow − outflow = next start stock; inflow only at Leh
War reserve	End-of-horizon depot stock + shortfall ≥ reserve (soft)
Corridor caps	West: I/III/V ≤ 6/26/2.5 per day; South: ≤ 2.5/11/1.2 per day; zero on closure days
Reachability	Variables exist only for pairs with a route in the current graph
Non-negativity	All stock, load, inflow and deficit variables ≥ 0

7.4 Solver configuration

Setting	Value
Solver	OR-Tools `pywraplp` with SCIP
Threads	1
Time limit	10,000 ms
Relative gap	1%
Objective sense	Minimise
Statuses surfaced	OPTIMAL, FEASIBLE, INFEASIBLE, ABNORMAL

A one-thread, 10-second configuration is deliberate for a demonstration: it keeps the model deterministic and prevents a pathological instance from hanging the UI. The recorded baseline and closure solves completed in 517–689 ms, well below the limit, but a larger network or a tighter gap could hit it. In that case the UI would show FEASIBLE and the objective would be the best incumbent, not a proof of optimality.

7.5 Plan outputs

A successful solve returns the status, objective, wall time, solver time, deliveries, gateway inflow, FOB and depot stock projections, FOB buffers and KPIs. A delivery row contains the day, source, destination, truck count, per-class loads, cost in rupees and route cost-km. The UI renders this as a dispatch table and uses the stock projection for the inventory chart. The closure endpoint embeds both the old and new plan payloads in the event so the diff can be audited later.

7.6 Baseline plan behaviour

> **Seeded 7-day baseline (recorded run)**
> Status OPTIMAL · 15 truck dispatches · 0 stockout units · 0 buffer-deficit units · objective ₹23.14 lakh. The plan uses both staging depots and the base depot, respects the daily fleet caps and keeps every FOB at or above its safety buffer across the horizon. The objective is dominated by procurement, not by the ₹4,000 dispatch fee.

The baseline is a plan for the 7-day operational horizon, not a 30-day guarantee. The 30-day evaluation is a separate simulation with its own rolling replans; its DRONA result is 0.25 stockout units, not zero. The report keeps those two claims separate on purpose.

7.7 Limitations

- **Deterministic demand.** The MILP uses the point forecast. It does not hedge against forecast error or weather uncertainty.
- **Unit aggregation.** A unit represents tonnes for classes I and V and kilolitres for class III. The model does not mix commodity-specific volume, weight or handling constraints.
- **No route scheduling.** A truck is available per day, but there is no vehicle routing, driver hours, convoy escort or road-window schedule.
- **Soft constraints.** Stockouts and buffer deficits are penalised, not forbidden. Extremely tight instances could choose a small stockout over a very expensive delivery; the penalty makes that rare but not impossible.
- **Time-limited optimality.** The 1% gap and 10-second limit mean the last decimal of the objective may vary across machines or instances.

8. Closure Response

8.1 Event pipeline

The closure endpoint is the most important integration path in the prototype. It must change the road network, the plan, the UI and the audit trail without allowing them to drift. The sequence is:

Step	Action	Failure handling
1	Validate the pass id and action	Unknown pass returns 404; invalid action is rejected by schema
2	Persist the closed-pass set	Survives subsequent plan requests
3	Rebuild the graph with the pass removed	West gateway becomes None if severed
4	Remap every depot→FOB and gateway route	Unreachable pairs are excluded from the MILP
5	Recompute corridor caps for the closure state	West inflow is zero while closed
6	Re-solve the 7-day MILP	Status and wall time recorded
7	Diff old and new deliveries	Cancelled, added, changed, unchanged counts
8	Persist the event with both plans and timings	Returned to the UI as one payload

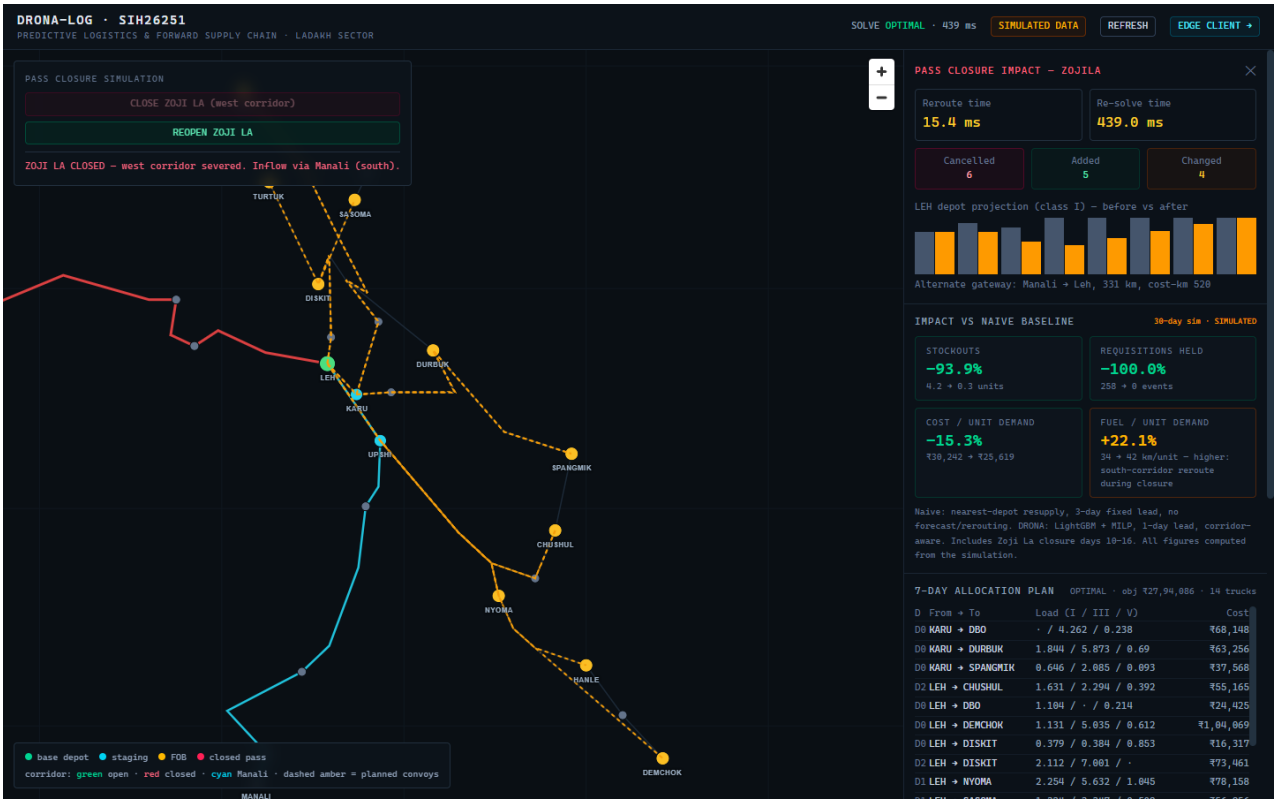


Figure 8.1 — Closure response in the command view. The west corridor is red, the alternate southern corridor is active, and the panel shows measured reroute/re-solve times, the plan diff and the depot inventory projection.

The screenshot is from the running prototype. The numbers on screen are live values from that run; they are not the same sample quoted in the event table below, because each solve measures its own wall time.

8.2 Recorded closure run

Item	Recorded value (20 Oct 2026)
Event	ZOJILA closed
Old plan → new plan	Plan 2 → Plan 3
Router time	28.5 ms

Item	Recorded value (20 Oct 2026)
Re-solve time	688.5 ms
Solver status	OPTIMAL
Objective	₹23,13,860.73 → ₹27,94,086.02 (+20.8%)
Trucks	15 → 14
West gateway route	None (severed)
South gateway route	MANALI → ROHTANG → KEYLONG → BARALACHA → SARCHU → DEBRING → TANGLANGLA → RUMTSE → UPSHI → LEH
South route distance / cost	331.36 km / 519.59 cost-km

The timings are wall-clock samples from one development machine. A second closure run produced 20.7 ms and 517.0 ms on reopen. The UI displays whatever the current solve returns; the report quotes samples to avoid implying a guaranteed latency.

8.3 Plan differences

Diff category	Count	Interpretation
Cancelled	6	Deliveries that existed before the closure and are no longer in the new plan
Added	5	New deliveries introduced by the new plan
Changed	4	Same day/source/destination but a different truck or class load
Unchanged	5	Deliveries that remain identical

A changed row is not a cosmetic edit. For example, a KARU→DBO day-0 delivery moved from 1.03 units of class III to 4.262 units of class III plus 0.238 units of class V. The new plan is re-optimised as a whole, not patched delivery by delivery, so quantities can move between classes and days to protect buffers under the reduced southern inflow.

8.4 Why the objective rises

The closure objective is 20.8% higher even though the new plan uses one fewer truck. Three forces dominate. First, procurement via the south corridor costs ₹24,000 per unit instead of ₹18,000, a 33% premium. Second, southern daily inflow caps are materially lower, so the plan cannot simply replace western tonnage one-for-one. Third, the longer southern route has a higher gradient penalty and reduces the set of feasible depot→FOB pairs. The optimiser is not failing to find a cheaper plan; it is finding the cheapest plan under a worse network and a more expensive supply source. Showing that increase is the honest outcome.

8.5 Operator experience

The command view deliberately couples the map and the numbers. The west corridor turns red, the southern corridor becomes the active route, planned convoys are redrawn from the new plan, and the impact panel shows both timings, the diff counts and the depot inventory projection. The panel is populated from the same response that produced the new plan, so there is no second fetch that could show a stale route. A 'REOPEN ZOJI LA' action restores the west corridor and re-solves again.

8.6 Reopening and multiple closures

The API persists a set of closed passes rather than a single boolean. Reopening removes the pass from that set and re-solves; closing another pass adds it. The demo exercises one pass at a time because that is the clearest operational story, and the tests verify the two named closure scenarios (Zoji La and Khardung La). A production deployment would need stress testing of simultaneous closures, partial reopenings and closure duration uncertainty; those are not claimed here.

9. Edge Operations and Ledger Integrity

9.1 Offline-first design

The edge client is a React view backed by IndexedDB. It does not depend on a service worker or a cached API response for its core function: requisitions are written directly to a local object store. Each entry records a device id, sequence number, previous hash, canonical payload, FOB, class, quantity, entry type, client timestamp and hash. The UI shows backend online/offline state, the local chain, pending versus synced entries, and the last sync result.

The ledger is append-only in normal operation. There is a deliberate 'clear local ledger' action for demonstrations; it is a reset, not a production feature. The local chain can be verified in the browser before sync, but the server verification is authoritative because the server recomputes the hash from the submitted fields and compares it with the submitted hash and its own chain head.

9.2 Hash-chained ledger

The hash covers both the chain fields and the semantic transaction fields, so changing a quantity, FOB, class or entry type by hand breaks the digest even if the JSON payload is left untouched:

```
body = seq | prev_hash | payload_json | fob_id | class_id | entry_type | qty.toFixed(4)
hash = SHA-256(body)

genesis: prev_hash = "0" × 64
canonical payload: {"fob": ..., "class": ..., "qty": ..., "note": ..., "ts": ...}
```

The browser uses WebCrypto `crypto.subtle.digest`; the server uses Python `hashlib.sha256`. A frontend test asserts byte-identical digests for the same input. The canonical payload is produced by a single function, so field order is stable and the two implementations hash the same string.

9.3 Sync verification

On `POST /api/sync` the server reads the per-device chain head from `chain_heads` (falling back to the latest accepted ledger row, or the genesis value for a new device). It sorts the batch by sequence and processes entries in order. For each entry it:

- recomputes the hash from the submitted chain and semantic fields;
- checks whether the entry id already exists, and if so, verifies that the stored copy and the submitted copy agree;
- requires `seq = previous + 1`;
- requires `prev_hash` to equal the stored chain head;
- rejects negative quantities;
- stops accepting later entries in the batch once the chain is broken;
- inserts accepted entries, advances the chain head and applies accepted receipts to stock.

The response reports accepted and rejected counts, a primary reason, the new chain-head sequence and per-entry results. The client marks only accepted entries as synced.

9.4 Rejection taxonomy

Condition	Server result	Why it matters
Hand-edited qty / FOB / class / type	HASH MISMATCH	The digest covers semantic fields, not only the JSON blob
Altered replayed entry with same id	HASH MISMATCH	Idempotency does not mean accepting a different payload
True duplicate with identical content	Accepted as duplicate, already synced	A retry after a timeout is safe
Sequence gap	Sequence gap error	A deleted or missing entry cannot be hidden
Broken previous link	`prev_hash` does not match preceding entry	Reordering or splicing is detected
Entry id owned by another device	Rejected	Prevents cross-device replay

Condition	Server result	Why it matters
Negative quantity	Rejected	Basic domain validation
Chain broken earlier in batch	Later entries rejected	One bad entry cannot be followed by silent acceptance

9.5 Demonstrated tamper rejection

The edge client includes a demonstration action that edits the last local entry's quantity without recomputing its hash. On the next sync the server returns '0 accepted · 1 rejected — HASH MISMATCH'. Entries already accepted on the server remain valid; the local chain is flagged as broken. The same behaviour is covered by tests for a tampered entry, a tampered payload, an altered replay, a sequence gap and a broken previous hash.

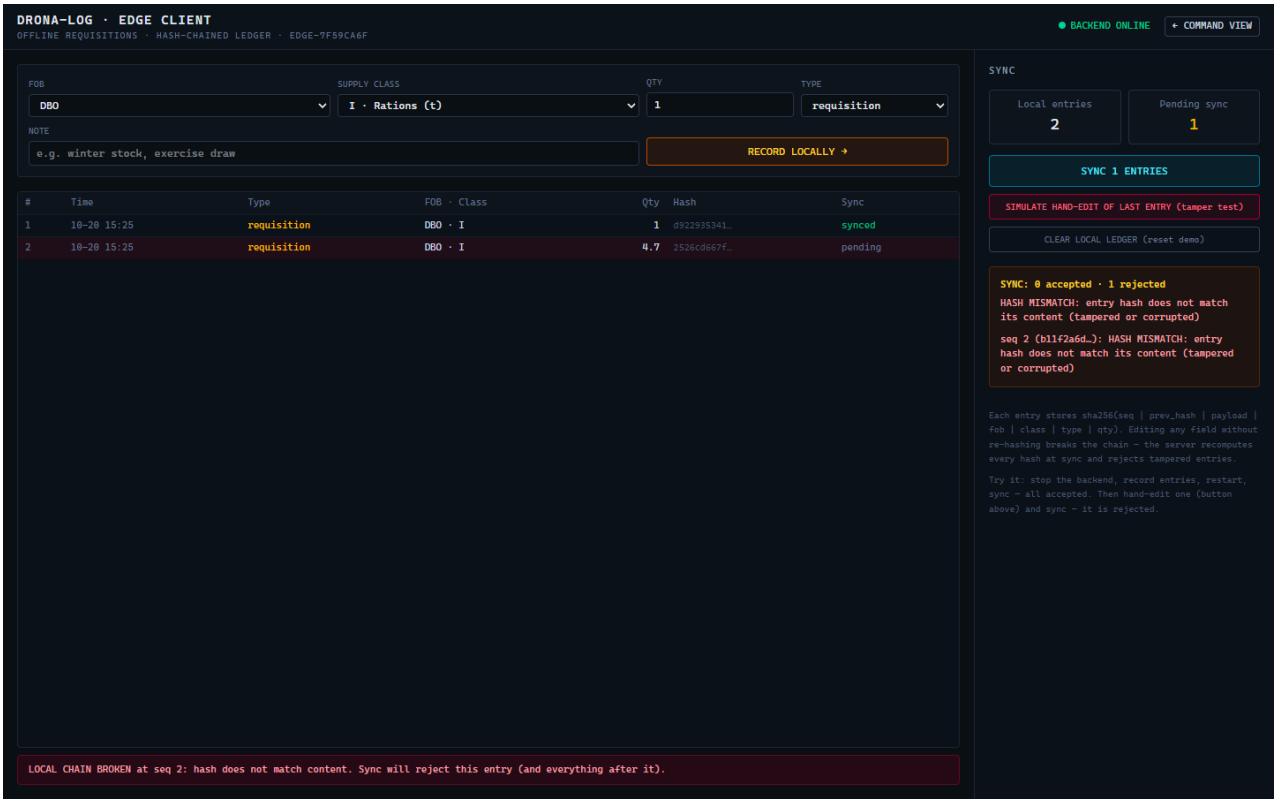


Figure 9.1 — Edge client after a hand edit and sync. The modified entry is rejected; the server reports zero accepted and one rejected with a hash mismatch.

9.6 Stock semantics and limitations

Only accepted receipt entries advance server stock. Requisitions are recorded and verified but do not silently change on-hand inventory; a requisition is a request, not a receipt. This distinction is visible in the code and is important for auditability: accepting a requisition as stock would create inventory that never physically arrived.

- **No device signing.** Possession of the device id and the ability to compute a hash is sufficient to submit a well-formed chain. Production needs registered device keys and signatures.
- **Fixed genesis.** Every device starts from 64 zeros. Production needs a device registration record bound to the genesis.
- **No conflict resolution.** Two devices can create independent chains; the server accepts each per-device chain but does not reconcile competing requisitions for the same stock.
- **Local reset.** The demo can clear the local ledger. This is not a tamper-proof hardware root of trust.
- **Browser storage.** IndexedDB can be cleared by the user or the browser under storage pressure. A production edge app needs durable storage and backup.

10. User Experience and Interface

10.1 Command view

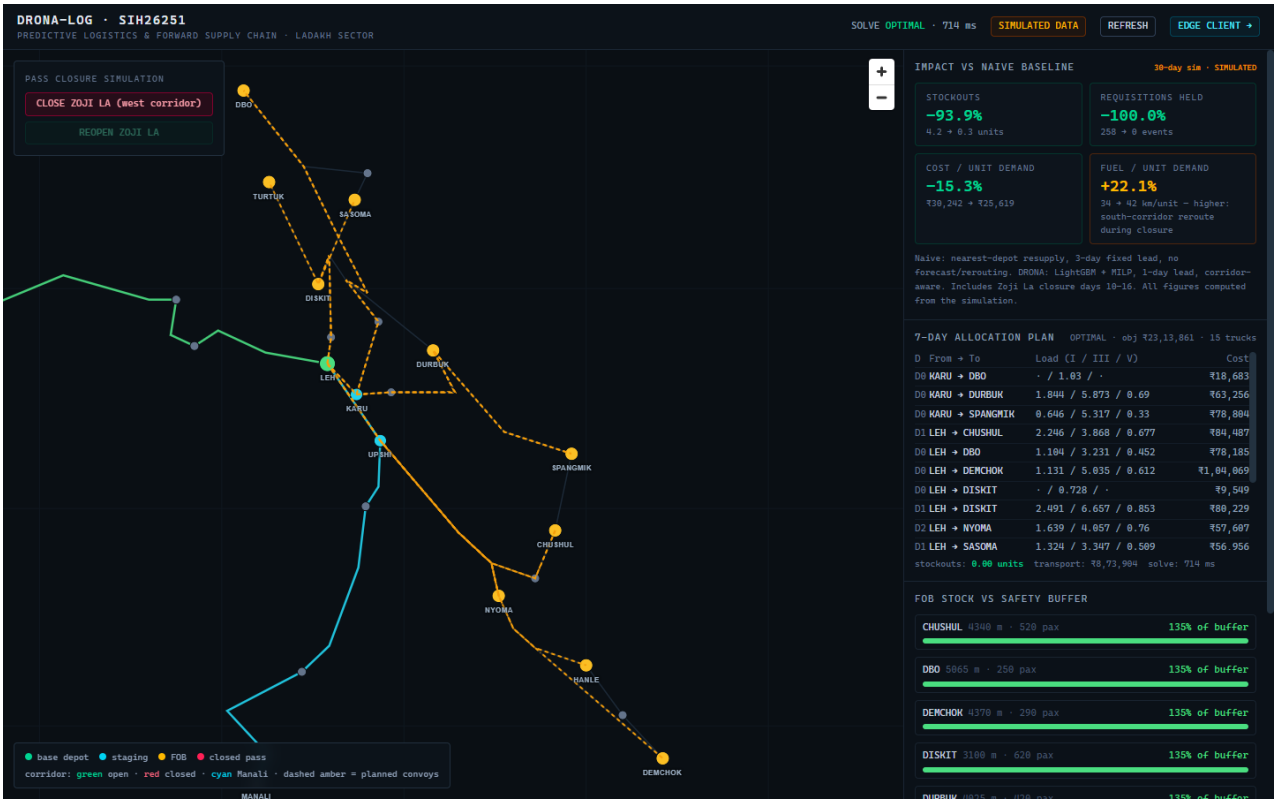


Figure 10.1 — Command view. The map, 7-day allocation table, stock-versus-buffer panel, closure controls and SIMULATED impact comparison share one screen.

The command view is built around a single question: what is the plan, and what happens if the road changes? The map answers the spatial part; the right pane answers the quantitative part. The closure control sits over the map where the corridor is visible, not buried in a settings menu. The SIMULATED impact panel is always present, so a planner sees the policy comparison without leaving the operational plan. The edge-client link is in the header, making the offline workflow discoverable.

10.2 Visual system

Element	Choice	Reason
Surface	Dark navy / charcoal panels	Reduces glare in a operations-room setting and keeps the map dominant
Open corridor	Green	Conventional 'go' signal
Southern corridor	Cyan	Distinguishes the alternate from the primary without implying warning
Planned convoy	Dashed amber	Visible over both corridors without competing with status colours
Closure / rejection	Red	Unambiguous exception state
Data provenance	Amber 'SIMULATED DATA' tag	Prevents synthetic figures being mistaken for live data
Typography	Compact sans-serif, uppercase micro-labels	High information density with clear grouping

The interface uses text labels in addition to colour: 'open', 'closed', 'synced', 'pending', 'accepted', 'rejected'. Colour is a fast cue, not the only carrier of meaning. The map legend spells out corridor and convoy states. This matters for both accessibility and for screenshots reproduced in black and white.

10.3 States and feedback

State	Interface behaviour
Loading	Plan panel shows a solving state; the closure button is disabled while a request is in flight
Backend offline	Edge client shows a red BACKEND OFFLINE indicator; requisitions still record locally
Sync pending	Pending entries are counted separately and marked 'pending'
Sync rejected	A red panel lists the rejected entry and the server reason
Plan infeasible	Status is surfaced from the solver rather than masked as success
Closure applied	West corridor turns red, alternate route is drawn, impact panel opens
Empty ledger	A short instruction replaces the table rather than showing an empty grid

10.4 Accessibility and usability notes

The UI has not been through a formal WCAG audit. Known gaps are the small 10–11 px micro-labels, a map that carries some information only spatially, limited keyboard navigation for map interactions and no screen-reader annotations for the canvas map. The design mitigates the most important issues with text labels, explicit status words and a plan table that repeats the map's operational content. A production pilot would need a contrast audit, larger type options and a keyboard-accessible alternative to the map.

11. Evaluation

11.1 Methodology and fairness controls

The evaluation compares two policies over the same 30-day seeded scenario. The naive policy is a constructed legacy baseline: nearest-depot resupply, a fixed three-day lead, a trailing seven-day mean for ordering, reorder points at the safety buffer, no forecasting and no rerouting. Convoys whose route crosses the closed pass are held, and gateway procurement waits for the west corridor. The DRONA policy uses the LightGBM forecast, a three-day rolling MILP replan, a one-day lead and corridor-aware rerouting; it replans on cadence, on a closure-state change and when a FOB falls below half its buffer.

Both policies start from the same stock and buffers, face the same 755 units of actual demand, the same weather and troop series, and the same seven-day Zoji La closure. Cost is divided by total demand rather than delivered units, so a policy cannot improve its cost per unit by failing to deliver. Truck-kilometres are divided by the same denominator. The comparison is recomputed by `POST /api/metrics/recompute` and cached; every card is labelled SIMULATED.

> **What is and is not comparable**
>
> The policies intentionally differ in lead time and replanning behaviour; that is the intervention being tested. They share the same cost constants, route distances, demand and starting stock. The naive baseline is a reasonable representation of a legacy reorder process, not a strawman with deliberately sabotaged parameters. Even so, it is a constructed baseline, not an observed unit's historical performance.

11.2 Results

Metric (30-day simulation)	Naive	DRONA-LOG	Change
Stockout units	4.15	0.25	−93.9%
Stockout rate (% of demand)	0.55%	0.034%	−93.9%
Below-buffer unit-days	983.9	493.3	−49.9%
Cost per unit demand	₹30,242	₹25,619	−15.3%
Total cost	₹2.28 crore	₹1.93 crore	−15.3%
Trucks dispatched	207	160	−22.7%
Truck-km per unit demand	34.46	42.06	+22.1%
Mean days of cover	8.6	8.2	−4.7%
Held-requisition counter	258	0	structural (see text)

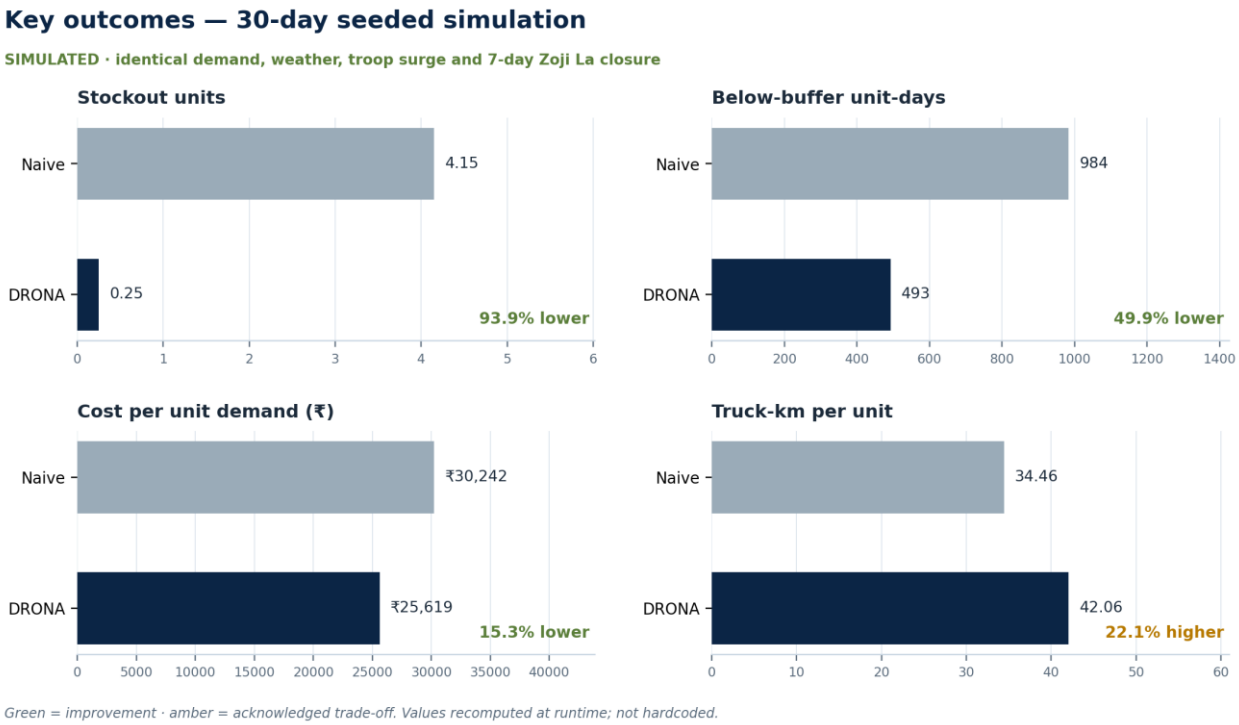


Figure 11.1 — Key 30-day outcomes. Green marks an improvement; amber marks the truck-km trade-off that DRONA-LOG accepts to maintain availability during the closure.

The total cost is the sum of transport and procurement. Naive transport is ₹76.06 lakh and procurement ₹1.52 crore; DRONA transport is ₹61.83 lakh and procurement ₹1.32 crore. The DRONA plan spends more per unit on southern procurement during the closure but buys and moves less overall, so total cost falls.

11.3 Interpretation and trade-offs

The headline is a 93.9% reduction in stockout units, from 4.15 to 0.25 units over 30 days. The absolute numbers are small because the scenario starts with meaningful safety stock, so the result should be read as a reduction in residual shortages under a closure, not as evidence that the theatre was on the brink of failure. The below-buffer metric is a broader measure of resilience: FOB-days below the safety buffer fall by 49.9%, which means the plan is not merely avoiding the visible stockout but keeping more formations inside their intended posture.

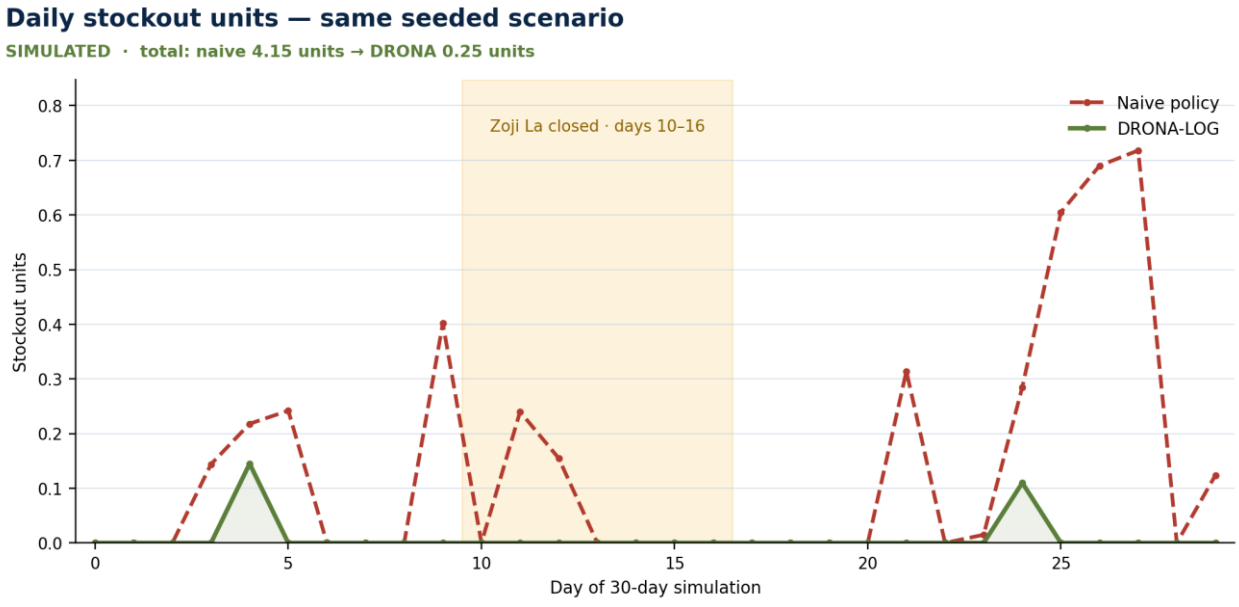


Figure 11.2 — Daily stockout units for both policies. The shaded band is the seven-day Zoji La closure. The DRONA policy absorbs the closure with two small residual events; the naive policy accumulates stockouts before, during and after it.

Cost per unit falls 15.3% because the DRONA policy carries less excess inventory and dispatches 47 fewer trucks, even though it pays a southern procurement premium during the closure. The truck-km result goes the other way: delivering through the longer southern corridor raises truck-km per unit by 22.1%. If truck-kilometres are used as a diesel proxy, DRONA-LOG consumes more fuel per unit of demand in this scenario. The system reports that as a warning, not a benefit. A planner can decide whether availability during a closure is worth the fuel; the tool does not make that value judgement for them.

The held-requisition counter requires a separate caveat. The naive policy increments the counter whenever a reorder is triggered but no load is available or a convoy is held at a closed pass. The DRONA simulation has no equivalent hold path, so its counter is zero by construction. The 258 → 0 row therefore illustrates the absence of the naive behaviour; it is not an independently measured 100% reduction in real held requisitions. The report and the UI both say so.

11.4 Variability

The seeded data and model training are deterministic, so the demand and forecast inputs reproduce on the same machine. The MILP is time-limited and the simulation performs many solves, so the last decimal of an objective or a delivery quantity can vary with solver version, host speed and thread scheduling. The metric cards are recomputed from the simulation at runtime rather than baked into the frontend. The values in this report are from the recorded run on 20 October 2026 and may differ slightly on another machine; the direction and approximate magnitude are the claim, not the last digit.

11.5 Threats to validity

- **Synthetic data.** The demand generator encodes assumed relationships. The measured improvement is evidence that the decision loop responds correctly to a controlled scenario, not a forecast of field performance.
- **Single scenario.** One 30-day window, one closure, one seed. There is no Monte Carlo distribution or sensitivity sweep across closure durations.
- **Constructed baseline.** The naive policy is a plausible legacy process, but it is not calibrated to a specific unit's historical behaviour.
- **Model mismatch.** The DRONA simulation uses a three-day rolling MILP replan, while the command view exposes a seven-day plan. This is intentional for compute cost, but it means the 30-day result is not a direct replay of the displayed plan.
- **Cost model.** Transport is linear in cost-km and procurement is a flat per-unit price. Real contracts, fuel surcharges, maintenance and crew costs are not modelled.
- **No stochastic optimisation.** Forecast error and weather uncertainty are not explicitly hedged; safety buffers and penalties are the only risk controls.

12. Acceptance Criteria and Evidence

12.1 Criterion-by-criterion evidence

Criterion	Implementation	Evidence and observed result
Feasible baseline with zero stockouts	MILP with 7-day horizon, fleet and stock constraints, soft stockout/buffer penalties	`test_baseline_plan_feasible_zero_stockouts`, `test_plan_zero_stockouts`; seeded plan OPTIMAL, 0 stockout units, 15 deliveries
Closure produces a valid alternate and a new plan	Closed pass removed from graph; A* remaps every depot→FOB route; MILP re-solves; diff and timings returned	`test_zojila_closure_severs_west_corridor`, `test_pass_closure_flow`; west route None, south route valid, objective rises
Solve time displayed	Wall-clock solve time stored in every plan and closure event	`solve_time_ms` and `re_solve`; UI shows 28.5 ms / 688.5 ms in the recorded closure
Offline entries sync after restart	IndexedDB append-only ledger; server idempotent sync; receipts applied	`test_sync_after_backend_restart`, `test_sync_receipt_updates_stock`; live demo 2 entries recorded offline and accepted after restart
Hand-edited entry rejected	Server recomputes SHA-256 over canonical fields and checks sequence and previous link	`test_tampered_entry_rejected`, `test_tampered_payload_rejected`, `test_tampered_replay_rejected`; UI 0 accepted, 1 rejected
One-command bring-up	Docker Compose with backend healthcheck, nginx SPA and `/api` proxy, persistent volume	`docker compose up --build`; health verified from 8080 and 8000
Metrics computed, not hardcoded	Naive and DRONA simulated on the same seed; cards labelled SIMULATED	`test_metrics_computed_not_hardcoded`, `test_drona_beats_naive_on_stockouts_and_holds`; live cards match counters
Run steps and demo documented	README, this report, ideas template, recorded demo, submission answers	Repository docs; 2:33 demo; 6-page PDF

12.2 Test and demonstration inventory

The acceptance evidence comes from three sources: automated tests, the running UI, and recorded events in the database. Each source covers a different risk. Tests prove invariants and rejection paths; the UI demonstrates that the integrated screen behaves as described; events prove that timings, plans and diffs were persisted in a real run.

Evidence source	Scope	Location
Backend tests	24 tests across API, planning, metrics and sync	`backend/tests/`
Frontend tests	6 hash/chain tests	`frontend/src/edge/ledger.test.js`
Recorded events	3 closure/reopen events with plans and timings	`GET /api/events`; `submission/events.json`
Screenshots	Command view, closure panel, tamper result	`docs/*.png`
Demo video	2:33 narrated walkthrough	`docs/drona-log-demo.mp4`
Idea template	6-page submission deck	`docs/Subterra_Mesh_SIH26251_Idea_Template.pdf`

13. Testing and Quality Assurance

13.1 Strategy

The test suite is organised around the failure modes that would invalidate the demonstration: an infeasible plan, a closure that leaves a severed route usable, a metric that is hardcoded, a sync that accepts a tampered entry, and a chain that tolerates a gap. Backend tests use FastAPI's test client against a seeded temporary database, so they exercise the real database and solver paths rather than mocks. Frontend tests exercise the ledger hash and chain functions in isolation; a parity test compares the JavaScript digest with a known Python digest.

13.2 Backend suite

File	Tests	Focus
`test_api.py`	7	Health, network shape, zero-stockout plan, forecast endpoint, closure flow, unknown pass, SIMULATED label
`test_planner.py`	6	Baseline feasibility, forecast shape, gradient penalty, Zoji La severance, Khardung La alternate, all-FOB reachability
`test_metrics.py`	3	Computed metrics, DRONA vs naive behaviour, series shapes
`test_sync.py`	8	Restart sync, receipt stock, tampering, payload tampering, idempotent replay, altered replay, sequence gap, broken link

The recorded run on 20 October 2026 completed 24 tests in 6.84 s, all passing. The exact wall time is machine-dependent; the pass count is the invariant.

13.3 Frontend suite

Test	What it checks
Same digest as backend	Byte-identical SHA-256 for identical canonical input
Hash changes when qty is edited	Quantity is covered by the digest
Well-formed chain verifies	Local chain walk accepts a valid sequence
Hand-edited entry detected	Qty changed without re-hashing is flagged
Swapped hash link detected	A broken previous link is flagged
Sequence gap detected	A missing sequence number is flagged

The recorded run completed 6 tests in 528 ms. The production bundle also builds successfully with `npm run build`.

13.4 Determinism and reproducibility

The seed, model parameters and one-thread solver configuration are fixed. The backend test database is seeded fresh for the suite. The frontend hash tests use fixed inputs. The reproduction checklist in Appendix D lists the exact commands and expected counts. A clean demonstration can be restored with `docker compose down -v` followed by `docker compose up -d`; this removes the volume, reseeds and retrains.

13.5 Coverage gaps

- **No browser end-to-end test.** The UI flows are verified by screenshots and manual demonstration, not by Playwright or Cypress.
- **No load or concurrency testing.** The prototype is single-user and single-node.
- **No solver stress suite.** Infeasible and time-limit paths are surfaced in code but not exercised by an automated test.
- **No security testing.** There has been no penetration test, dependency audit or threat-model review.
- **No accessibility audit.** The known gaps are listed in Section 10.4.
- **No multi-device conflict test.** The server supports per-device chains but does not reconcile competing requisitions.

14. Deployment and Operations

14.1 Container topology

The stack is defined in `docker-compose.yml` and runs as two services plus a named volume. The backend healthcheck is the gate for the frontend, which avoids the common failure where nginx serves the SPA before the API is ready and the first request fails.

Service	Port	Depends on	Notes
backend	8000	—	FastAPI, seed, LightGBM models, SQLite; healthcheck `/api/health`
frontend	8080	backend healthy	nginx SPA, `/api` proxy, static assets
drona-data	—	—	SQLite database and model artefacts

14.2 First boot

On first boot the backend seeds the database with 49 nodes, 53 edges, 365 days of history and 10 FOBs, then trains three LightGBM models. This takes roughly 30 seconds on a development laptop. The frontend waits for the healthcheck, so the first page load may show a short wait rather than a connection error. Subsequent starts reuse the persisted database and models.

```
docker compose up --build

# command view      http://localhost:8080
# edge client       http://localhost:8080/edge
# API documentation http://localhost:8000/docs
```

14.3 Local development

The backend and frontend can also run outside Docker. The backend uses a Python virtual environment, seeds explicitly and starts uvicorn on port 8000. The frontend uses Vite on 5173 and proxies `/api` to 8000. The seed command is idempotent and accepts `--force` to rebuild.

```
cd backend
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
python -m app.seed --force
python -m uvicorn app.main:app --port 8000

cd frontend
npm install
npm run dev          # http://localhost:5173
```

14.4 Reset and troubleshooting

Symptom	Likely cause	Resolution
Frontend shows connection error	Backend still seeding or training	Wait for healthcheck; check `docker compose ps` and backend logs
Plan is infeasible	Tight instance or corrupted stock	Recompute; reset volume and reseed if the demo state is corrupted
Models missing	Volume cleared or model files removed	Restart backend; it retrains if any model file is missing
Edge entries will not sync	Sequence gap, broken chain or backend offline	Read the sync reason; clear the local ledger only for a demo reset
Port already in use	Another process on 8000 or 8080	Stop the process or change the host port mapping

A clean demonstration state is `docker compose down -v && docker compose up -d`. The `-v` flag deletes the named volume, including ledger and plans; this is a demo reset, not a production operation.

14.5 Runtime dependencies

The runtime needs Docker with Compose, or Python 3.12+ and Node.js for a local run. There is no runtime API key, external tile server, CDN, database server or GPU. The build downloads base images and packages, but the demonstration itself is offline-capable. The backend exposes port 8000 for API inspection; the frontend exposes 8080.

15. Security, Ethics and Data Handling

15.1 Current security posture

Control	Status in prototype
Input validation	Pydantic models for sync and closure payloads; pass existence checked; negative quantities rejected
SQL injection	Parameterised SQLite queries throughout the backend
Transport	Local HTTP only; TLS terminates nowhere in the prototype
Authentication / roles	Not implemented
Device identity	Device id is client-supplied; no registration or signature
Secrets	No runtime API keys or credentials are used
Data at rest	SQLite file in a Docker volume; no encryption
Audit	Plans, closure events and sync results are persisted; no administrator audit UI

The prototype should be treated as a controlled demonstration, not a system for operational data. It is not hardened for a contested network, multi-user access or classified information.

15.2 Gaps before production

- **Authentication and role-based access.** A commander, planner and edge quartermaster need distinct permissions and an auditable identity.
- **Device registration and signing.** A hash chain proves internal consistency, not authorship. Entries should be signed by a registered device key.
- **TLS and network segmentation.** The API should not be exposed over plain HTTP outside a trusted host.
- **Secrets and key management.** Model artefacts, device keys and signing keys need a managed lifecycle.
- **Data classification.** The system must be accredited for the classification of the data it handles, with appropriate storage and logging controls.
- **Rate limiting and DoS protection.** Sync and solve endpoints are expensive and currently unthrottled.
- **Dependency and container scanning.** No SBOM, image scan or patch process is in place.

15.3 Data ethics and responsible claims

All data is synthetic. No personal data is collected, no classified movement record is used and no live unit is represented. The map is schematic and must not be treated as an operational map. The forecast is a decision-support output, not an order; a human planner remains responsible for the plan. The evaluation is presented with its trade-offs and its structural artefacts (the hold counter) rather than as a marketing claim.

The team has deliberately avoided claims that the prototype cannot support: it does not claim fuel savings, real-world forecast accuracy, production security, or a measured reduction in actual held requisitions. Those distinctions are part of the submission's honesty requirement, not an afterthought.

16. Limitations and Roadmap

16.1 Current limitations

Area	Limitation	Consequence
Data	Synthetic weather, demand and network; schematic geometry	External validity is unproven
Forecast	Point estimate; pooled per class; no drift monitoring	No service-level or uncertainty-aware planning
Routing	Static gradients; no traffic, surface or seasonal road state	Travel time and risk are approximated
Optimisation	Deterministic demand; unit aggregation; no vehicle scheduling	Plan is a capacity plan, not a dispatch schedule
Evaluation	One seed, one 30-day window, one closure; constructed baseline	No statistical confidence interval
Edge	No device signatures, conflict resolution or durable hardware root	Trust is server-side consistency, not authorship
Security	No auth, TLS, roles or accreditation	Demonstration only
UI	Desktop-first; no formal accessibility audit; canvas map	Limited field usability

16.2 Production roadmap

Phase	Focus	Exit criterion
0–3 months	Real data adapters (weather, road status, stock), configurable theatre, signed device identities	A pilot theatre can be loaded and a signed edge entry verified end to end
3–6 months	Stochastic or robust demand, multi-pass stress tests, browser E2E tests, authentication and roles	Closure scenarios pass an automated end-to-end suite with authenticated users
6–12 months	Pilot with logistics users, calibration of cost and gradient weights, monitoring and retraining	Measured pilot metrics against the legacy process, with a documented rollback

The roadmap is intentionally staged. Adding real data and authentication before the decision loop is validated would obscure whether the improvement comes from the model or from the data. The prototype's role is to prove the loop, the evidence trail and the operator experience; the pilot's role is to test them against reality.

16.3 What would make the evaluation stronger

- A Monte Carlo sweep over seeds, closure durations and demand shocks, reporting distributions rather than one run.
- A calibrated naive baseline derived from historical unit behaviour rather than a constructed policy.
- Forecast backtests with rolling origins and error metrics by class and FOB.
- Solver sensitivity to the 1% gap, time limit and penalty weights.
- Usability testing with logistics staff, including a map-free fallback and larger type.

17. Conclusion

DRONA-LOG demonstrates a complete, inspectable decision loop for forward supply in a mountain theatre. It forecasts demand per supply class, routes over terrain with an explicit gradient penalty, allocates a capacitated 7-day plan with a mixed-integer optimiser, and replans in under a second of solve time when a pass closes. It keeps requisitions working offline and gives the server a way to verify that they were not altered. The same API payload drives the map, the plan, the diff and the timings, so the operator sees one coherent story.

The measured result is meaningful but bounded: on a seeded 30-day scenario with a seven-day closure, stockouts fall by 93.9%, below-buffer unit-days by 49.9% and cost per unit of demand by 15.3%. Truck-kilometres per unit rise by 22.1% because availability is maintained through a longer corridor. The report keeps that trade-off visible, along with the structural nature of the zero hold counter. The prototype also passes 24 backend and 6 frontend tests, builds a production bundle and comes up with one command.

The next step is not a larger slide deck; it is a pilot with real weather and road data, signed device identities, authenticated users and a calibrated baseline. DRONA-LOG is ready to be tested in that setting because its assumptions, costs, failure modes and evidence are explicit. That is the standard the submission sets for itself.

References

[1] Team Subterra Mesh. DRONA-LOG repository, SIH26251 prototype. github.com/athul-dev-sys/DRONA-LOG (accessed 20 October 2026).

[2] FastAPI. Documentation. https://fastapi.tiangolo.com (accessed 20 October 2026).

[3] LightGBM. Documentation. https://lightgbm.readthedocs.io (accessed 20 October 2026).

[4] Google OR-Tools. Documentation and SCIP solver. https://developers.google.com/optimization (accessed 20 October 2026).

[5] NetworkX. Documentation. https://networkx.org (accessed 20 October 2026).

[6] MapLibre GL JS. Documentation. https://maplibre.org (accessed 20 October 2026).

[7] MDN Web Docs. IndexedDB API and Web Crypto API. https://developer.mozilla.org (accessed 20 October 2026).

[8] Smart India Hackathon. Official portal. https://www.sih.gov.in (accessed 20 October 2026).

Appendix A — API Reference

Method	Endpoint	Purpose
GET	`/api/health`	Seed, model and metrics readiness
GET	`/api/network`	Nodes, edges, stock, gateway routes and corridor state
GET	`/api/fobs`	FOB list with stock and demand context
GET	`/api/forecast?days=7`	Per-FOB, per-class LightGBM forecast
GET	`/api/plan`	Latest 7-day MILP plan; solves on first request
POST	`/api/plan/recompute`	Force a fresh solve
POST	`/api/events/pass-closure`	Close or reopen a pass; reroute, re-solve, diff, time
GET	`/api/events`	Closure and reopen audit log
GET	`/api/metrics`	SIMULATED naive-vs-DRONA comparison
POST	`/api/metrics/recompute`	Recompute the 30-day comparison
POST	`/api/sync`	Verify and ingest edge ledger entries
GET	`/api/ledger`	Server ledger rows in sequence order

The closure payload is `{ "pass_id": "ZOJILA", "action": "closed" | "reopened", "days": 7 }`. Interactive schemas are available at `/docs` when the backend is running.

Appendix B — Configuration Parameters

Parameter	Value	Meaning
SIM_TODAY	2026-10-01	Simulation start date
EVAL_DAYS	30	Evaluation horizon
FORECAST_DAYS / PLAN_HORIZON_DAYS	7 / 7	Operational forecast and MILP horizons
GRADIENT_K	6.0	Cost-km added per 1% gradient
ROAD_FACTOR	1.25	Road distance vs great-circle
TRUCK_CAPACITY	10.0	Units per truck
RATE_PER_UNIT_KM	₹42	Variable transport cost
TRUCK_FIXED_COST	₹4,000	Fixed dispatch cost per truck
FLEET	24 / 8 / 8	Trucks per day at Leh / Karu / Upshi
STOCKOUT_PENALTY	₹500,000	Penalty per unmet unit
BUFFER_PENALTY	₹60,000	Penalty per unit-day below buffer
INFLOW_WEST_CAP	6 / 26 / 2.5	West units/day for classes I / III / V
INFLOW_SOUTH_CAP	2.5 / 11 / 1.2	South units/day for classes I / III / V
PROC_COST_WEST / SOUTH	₹18,000 / ₹24,000	Procurement cost per unit
DELIVERY_LEAD_DAYS	1	Dispatch-to-arrival lead
SOLVE_TIME_LIMIT_MS	10,000	SCIP time limit
SOLVE_GAP_REL	0.01	SCIP relative gap

Appendix C — Test Inventory

C.1 Backend (24 tests)

Test	Invariant
test_health	Health endpoint reports seeded/ready state
test_network_shape	49 nodes and 53 edges, required FOBs and passes
test_plan_zero_stockouts	Baseline plan has zero stockout units
test_forecast_endpoint	Forecast shape and non-negative daily values
test_pass_closure_flow	Closure returns new plan, diff and timings
test_unknown_pass_404	Unknown pass is rejected
test_metrics_simulated_label	Metrics payload carries SIMULATED
test_metrics_computed_not_hardcoded	Metrics match simulation counters
test_drona_beats_naive_on_stockouts_and_holds	Directional policy comparison
test_series_shapes	Daily series lengths match the 30-day horizon
test_baseline_plan_feasible_zero_stockouts	MILP feasibility and zero stockout
test_forecast_shape	Per-FOB per-class forecast shape
test_router_gradient_penalty	Gradient penalty is applied to edge cost
test_zojila_closure_severs_west_corridor	West route is None after Zoji La closure
test_khardungla_closure_alternate_route	Nubra traffic routes via Wari La
test_all_fobs_reachable_baseline_and_closure	Every FOB reachable in both states
test_sync_after_backend_restart	Offline entries accepted after restart
test_sync_receipt_updates_stock	Accepted receipt advances stock
test_tampered_entry_rejected	Hand-edited quantity rejected
test_tampered_payload_rejected	Altered payload rejected
test_replay_is_idempotent	True duplicate accepted idempotently
test_tampered_replay_rejected	Altered replay rejected
test_sequence_gap_rejected	Sequence gap rejected
test_broken_prev_hash_rejected	Broken previous link rejected

C.2 Frontend (6 tests)

Test	Invariant
Same digest as backend	JavaScript and Python hashes match byte-for-byte
Hash changes when qty is edited	Quantity is bound to the digest
Well-formed chain verifies	A valid local chain passes
Hand-edited entry detected	Qty change without re-hash fails
Swapped hash link detected	Broken previous link fails
Sequence gap detected	Missing sequence number fails

Appendix D — Reproducibility Checklist

Step	Action
1	Clone the repository at the submitted revision.
2	Run `docker compose up --build` from the repository root.
3	Wait for the backend healthcheck and the first-boot seed/training to complete.
4	Open `http://localhost:8080` and confirm the SIMULATED banner and the 7-day plan.
5	Open `http://localhost:8080/edge` and record two requisitions.
6	Stop the backend with `docker compose stop backend`; confirm BACKEND OFFLINE and record another entry.
7	Start the backend with `docker compose start backend`; wait for BACKEND ONLINE and press SYNC.
8	Press SIMULATE HAND-EDIT OF LAST ENTRY, then SYNC; confirm one rejection with HASH MISMATCH.
9	Return to the command view and close Zoji La; confirm the red west corridor and the new timings/diff.
10	Reopen Zoji La; confirm the west corridor is restored.
11	Run `cd backend && python -m pytest tests/ -q`; expect 24 passed.
12	Run `cd frontend && npx vitest run`; expect 6 passed.
13	Run `cd frontend && npm run build`; expect a successful production build.
14	Optional reset: `docker compose down -v && docker compose up -d`.

Appendix E — Submission Artefacts

Artefact	File	Detail
Detailed report	`docs/DRONA-LOG_Detailed_Report.pdf`	This document; PDF and editable DOCX
Idea template	`docs/Subterra_Mesh_SIH26251_Idea_Template.pdf`	6 pages; 1.26 MB; below the 10 MB limit
Editable deck	`submission/final/Subterra_Mesh_SIH26251.pptx`	Final 6-slide source used for the template PDF
Demo video	`docs/drona-log-demo.mp4`	2:33 narrated walkthrough; 5.6 MB
Submission answers	`docs/SUBMISSION_ANSWERS.md`	Title, abstract, description and technology bucket
Repository	`github.com/athul-dev-sys/DRONA-LOG`	Source, tests, Docker Compose, README

Portal field summary: Idea Title 87 characters; Abstract 2,687 characters; Idea Description 11,450 characters; Technology Bucket Software; Idea Template PDF 1.26 MB. The full copy-paste text is in `docs/SUBMISSION_ANSWERS.md`.

— End of report —